

# DVD-JEPA: A Minimal, Reproducible Joint-Embedding Predictive Architecture World Model

Mandar Wagh<sup>1</sup>

<sup>1</sup>Black Robotics    [roboticsblack@gmail.com](mailto:roboticsblack@gmail.com)

June 2026

Code: [github.com/mandarwagh9/dvd-jepa](https://github.com/mandarwagh9/dvd-jepa) · Interactive demo: [dvd-jepa.vercel.app](https://dvd-jepa.vercel.app)

## Abstract

A *world model* predicts how an environment evolves. The dominant failure mode when learning one from video is generative: predicting future pixels forces the model to spend capacity on detail that is fundamentally unpredictable. Joint-Embedding Predictive Architectures (JEPA) instead predict the *representation* of the future and let the encoder discard what it cannot predict. We present **DVD-JEPA**, the smallest honest instance of this idea we could construct: a context encoder, an exponential-moving-average (EMA) target encoder, and a latent predictor are trained—without labels and without a decoder—to predict the next observation of a bouncing DVD logo in a 32-dimensional representation space. Despite never receiving a coordinate, a frozen linear probe recovers the logo’s position to 0.81 px in a 16 px box. An ablation shows that removing the stop-gradient collapses the representation to a constant, while the EMA target prevents it. Adding an optional decoder turns the model into a future-frame video predictor that tracks the bounce for  $\sim 20$  steps before latent rollout drift, and into a predictive anomaly detector whose surprise signal spikes  $55\times$  over baseline on an injected teleport. The entire model trains on a CPU in  $\sim 10$  seconds and the trained weights run client-side in a browser. DVD-JEPA is deliberately a toy, but every component has a full-scale counterpart in I-JEPA, V-JEPA, and V-JEPA 2; it is meant as a transparent, runnable artifact for understanding how JEPA world models work.

## 1 Introduction

A world model answers a single question: given the current state and (optionally) an action, what happens next? Animals build such models from passive observation, and learning them from video is a central goal of self-supervised representation learning [1, 7]. The obvious approach is *generative*—predict the next frame pixel by pixel—but this is the wrong objective. Most of the future is unpredictable in its fine detail (exact textures, noise) yet entirely predictable in the abstract (an object keeps moving). A pixel-predicting model is forced to model the irreducible noise and produces blurry, hedged predictions.

The Joint-Embedding Predictive Architecture (JEPA) makes a different bet [1, 2]: predict in representation space, not pixel space, and let the encoder make itself invariant to whatever it cannot

predict. This is the principle behind I-JEPA for images [2], V-JEPA for video [3], and V-JEPA 2, which adds action-conditioning to drive real-robot planning [4].

These systems are large and trained on enormous corpora, which makes the core mechanism hard to inspect. We take the opposite extreme. **DVD-JEPA** reduces the idea to its smallest faithful form: the world is a DVD logo bouncing in a  $16 \times 16$  box, the model is three small multilayer perceptrons, and the whole thing trains on a CPU in about ten seconds. Our contributions are:

- A minimal, fully reproducible JEPA world model with the complete anti-collapse machinery (EMA target encoder, stop-gradient, variance regularization).
- Experiments isolating each claim: the latent encodes exact world state (linear probe), the stop-gradient is necessary to avoid collapse (ablation), the latent rollout forecasts the future and then drifts (horizon curve), and the rendered prediction yields a usable anomaly signal.
- An interactive, client-side browser demo and a Colab notebook, so the architecture can be understood by manipulation rather than by reading alone.

## 2 Background

**Three architectures.** Given two related inputs  $x$  and  $y$ , one can (i) jointly embed them and match the embeddings (Siamese / joint-embedding architectures), (ii) generate  $y$  from  $x$  in input space (autoencoders, masked autoencoders), or (iii) embed both *and* predict one embedding from the other. The third is JEPA: an encoder produces  $s_x, s_y$  and a predictor outputs  $\hat{s}_y = P(s_x, z)$ , with the loss  $D(\hat{s}_y, s_y)$  computed in representation space [1]. Because the loss never touches pixels, the target encoder is free to drop unpredictable detail.

**Collapse.** The trivial solution is for both encoders to output a constant, making the loss zero. JEPA-style methods prevent this with asymmetry: the target encoder is an EMA of the online encoder with a stop-gradient (the BYOL mechanism [6]), optionally reinforced by information regularization such as the variance/covariance terms of VICReg [5]. We use both and ablate them in Section 4.

**From JEPA to a world model.** Making the predictor advance one time step turns it into a transition model in representation space; conditioning it on actions makes it a controllable world model that can be used for planning in latent space, as in V-JEPA 2 [4]. DVD-JEPA has no actions (the logo bounces autonomously), so its predictor is an unconditioned one-step transition model.

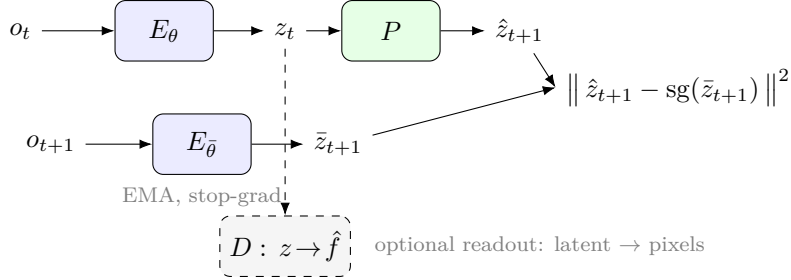
## 3 The DVD-JEPA Model

**The world.** A DVD logo bounces at constant speed in a  $16 \times 16$  box, reflecting off the walls and rendered as a soft Gaussian blob. This is the simplest system with the property that matters: the future is unreadable from a single frame (a static dot has no visible velocity) but perfectly predictable from two. An *observation* is therefore a stack of two consecutive frames,  $o_t = (f_t, f_{t+1}) \in \mathbb{R}^{2 \cdot 16 \cdot 16}$ .

**Architecture.** Three MLPs (Table 1, Figure 1): a context encoder  $E_\theta$ , an EMA target encoder  $E_{\bar{\theta}}$ , and a predictor  $P$ . An optional decoder  $D$  maps a latent back to a frame; a *pure* JEPA omits it.

| Component                         | Mapping                 | Shape                                                | Role                             |
|-----------------------------------|-------------------------|------------------------------------------------------|----------------------------------|
| Context encoder $E_\theta$        | $o \rightarrow z$       | $512 \rightarrow 256 \rightarrow 128 \rightarrow 32$ | encode an observation            |
| Target encoder $E_{\bar{\theta}}$ | $o \rightarrow \bar{z}$ | EMA of $E_\theta$ , stop-grad                        | produce the prediction target    |
| Predictor $P$                     | $z \rightarrow \hat{z}$ | $32 \rightarrow 64 \rightarrow 32$                   | one step forward in latent space |
| Decoder $D$ (optional)            | $z \rightarrow f$       | $32 \rightarrow 64 \rightarrow 256 \rightarrow 256$  | render a latent to pixels        |

**Table 1:** The three (plus one) small networks. The entire forward pass is a handful of matrix multiplies, which is what lets it run client-side in  $\sim 40$  lines of JavaScript.



**Figure 1:** DVD-JEPA. The predictor matches the EMA target encoder’s embedding of the next observation, entirely in latent space (sg is stop-gradient). The decoder is an optional readout used only to visualize and to compute a pixel-space anomaly score.

**Objective.** With observations  $o_t$  and  $o_{t+1}$ , we minimize the latent prediction error plus a VICReg variance term over a batch of online embeddings  $Z$ :

$$\mathcal{L} = \|P(E_\theta(o_t)) - \text{sg}(E_{\bar{\theta}}(o_{t+1}))\|^2 + \sum_{d=1}^{32} \max(0, 1 - \sigma(Z_{\cdot,d})), \quad (1)$$

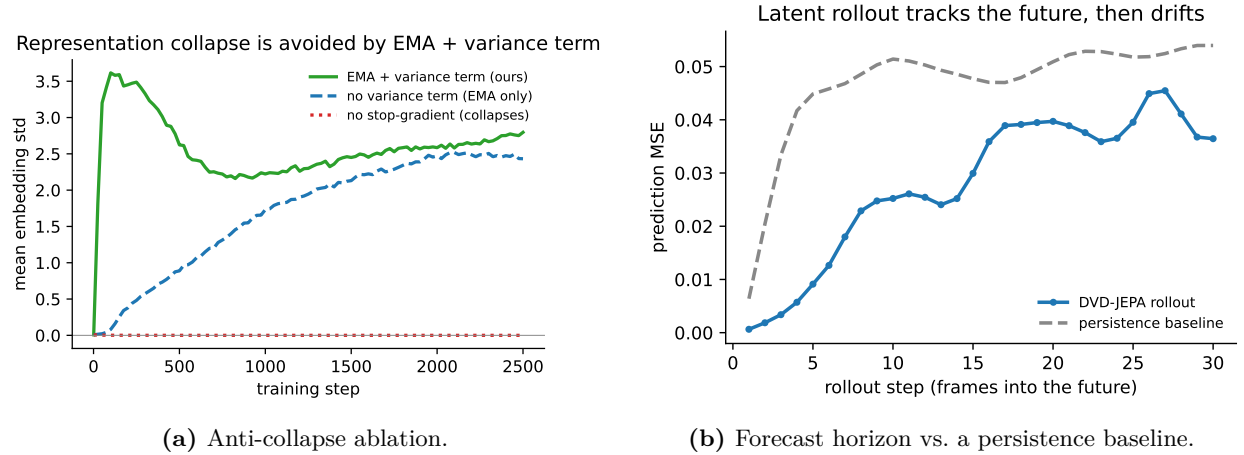
where  $\sigma(\cdot)$  is the per-dimension standard deviation and sg is stop-gradient. The target encoder is updated as  $\bar{\theta} \leftarrow \tau \bar{\theta} + (1 - \tau)\theta$  with  $\tau = 0.99$ . The decoder, when used, is trained *separately* on the frozen encoder, so the JEPA performs all of the representation learning.

## 4 Experiments

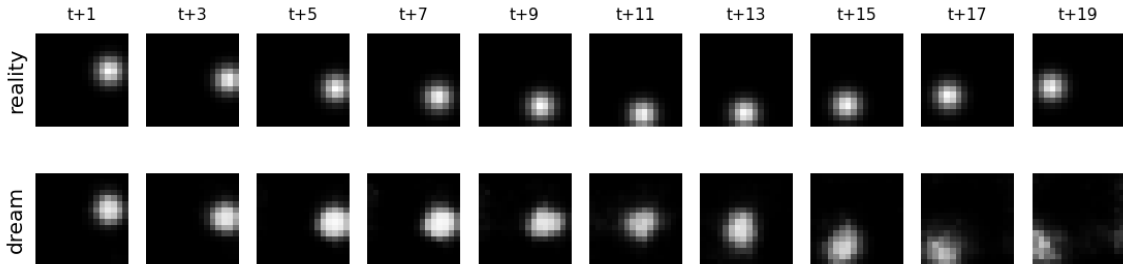
All results use a fixed seed and run on a CPU in  $\sim 10$  seconds for the core model; figures are regenerated by `paper/figures.py`.

**The representation encodes world state.** We freeze the encoder and fit a *linear* map from the 32-d latent to the true  $(y, x)$  position. It achieves 0.81 px RMSE on held-out data in a 16 px box—the latent encodes the exact state of the world despite the model never being shown a coordinate or a reconstruction target.

**The stop-gradient prevents collapse.** Figure 2a tracks the mean embedding standard deviation during training for three variants. Removing the stop-gradient (making the target trainable) collapses the representation to a constant ( $\sigma \rightarrow 0$ ). The EMA target alone keeps the representation healthy; the variance term is a safety margin that pins the scale.



**Figure 2:** Left: removing the stop-gradient collapses the representation; the EMA target prevents it. Right: the latent rollout tracks the future far better than persistence, then drifts as errors compound.



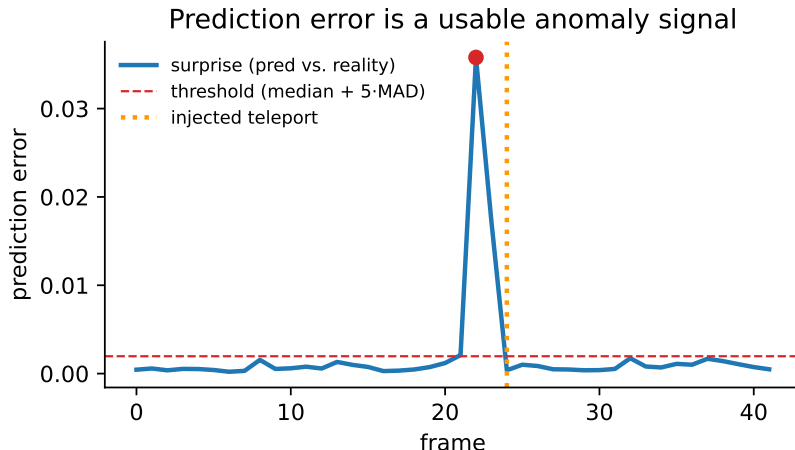
**Figure 3:** Future-frame prediction. Top: reality. Bottom: the decoded latent dream, rolled forward in representation space. It reproduces the trajectory and the wall bounce for roughly twenty steps.

**The latent rollout forecasts, then drifts.** We seed from two frames and roll the predictor forward in latent space, decoding each step. Figure 2b compares the decoded rollout against a persistence baseline (hold the last observed frame). The model is far more accurate for the near horizon and degrades gracefully; the filmstrip (Figure 3) shows it correctly imagining the bounce for  $\sim 20$  steps before single-step errors compound. Mean prediction MSE rises from 0.0006 at one step to 0.036 at 30 steps.

**Prediction error is a usable anomaly signal.** Run as a one-step monitor, the model renders what it expects next; when reality diverges, the pixel error spikes. We inject a teleport at frame 24; the surprise peaks  $55\times$  over baseline and is flagged at frame 22 (Figure 4), two frames early because the monitor predicts two steps ahead.

## 5 Interactive Demo

The trained networks are exported as base64 float32 and re-implemented in dependency-free JavaScript, so the model runs entirely client-side at [dvd-jepa.vercel.app](https://dvd-jepa.vercel.app) (no server, no GPU). A viewer can toggle the decoder off to see that a *pure* JEPA returns only a 32-vector and cannot draw; inject anomalies and watch the surprise meter; and roll the predictor forward to watch multi-step latent drift. We find that manipulating the model in real time conveys the predict-in-representation-space



**Figure 4:** DVD-JEPA as a predictive anomaly detector. A teleport injected at frame 24 produces a surprise spike  $55\times$  over baseline, crossing the threshold.

idea more directly than static figures.

## 6 Related Work

DVD-JEPA is a miniature of I-JEPA [2] and V-JEPA [3], which apply the same predict-in-latent-space objective with EMA target encoders at Vision-Transformer scale, and of V-JEPA 2 [4], which adds action-conditioning for planning. The anti-collapse machinery follows BYOL [6] and VICReg [5]. It contrasts with *generative* world models such as World Models [7] and Dreamer [8], which predict in pixel or reconstructed latent space and can render but must model irreducible detail. DVD-JEPA is, deliberately, the non-generative counterpart shrunk to a size one can hold in one’s head.

## 7 Limitations

The world is  $16 \times 16$  and deterministic, so there is no need for a stochastic latent to model multi-modal futures (real JEPAs include one). The predictor is trained for a single step, so autoregressive rollouts drift after  $\sim 20$  steps; multi-step or recurrent training would extend the horizon. The decoder is a crutch added only for visualization and pixel-space scoring—a pure JEPA has none. Finally, the demonstrations are qualitative-scale: the value of DVD-JEPA is pedagogical transparency, not a new state of the art.

## 8 Conclusion

DVD-JEPA shows that a complete, correct JEPA world model—representation-space prediction, EMA target, anti-collapse, latent rollout, and an optional rendering head—fits in a few hundred lines and trains on a CPU in seconds. It learns the physics of a bouncing logo with no labels, forecasts the future, and detects anomalies, while remaining small enough to inspect end to end and interactive enough to play with in a browser. We hope it serves as a runnable on-ramp to the full-scale systems it imitates.

**Reproducibility.** Code, trained weights, the interactive demo, and a Colab notebook are at [github.com/mandarwagh9/dvd-jepa](https://github.com/mandarwagh9/dvd-jepa). Running `python -m dvd_jepa.train` reproduces the model and `python paper/figures.py` reproduces every figure and number in this paper.

## References

- [1] Y. LeCun. *A Path Towards Autonomous Machine Intelligence*. Open Review, 2022.
- [2] M. Assran et al. *Self-Supervised Learning from Images with a Joint-Embedding Predictive Architecture (I-JEPA)*. CVPR, 2023.
- [3] A. Bardes et al. *Revisiting Feature Prediction for Learning Visual Representations from Video (V-JEPA)*. TMLR, 2024.
- [4] Meta AI. *V-JEPA 2: Self-Supervised Video Models Enable Understanding, Prediction and Planning*. 2025.
- [5] A. Bardes, J. Ponce, Y. LeCun. *VICReg: Variance-Invariance-Covariance Regularization for Self-Supervised Learning*. ICLR, 2022.
- [6] J.-B. Grill et al. *Bootstrap Your Own Latent (BYOL)*. NeurIPS, 2020.
- [7] D. Ha, J. Schmidhuber. *World Models*. NeurIPS, 2018.
- [8] D. Hafner et al. *Dream to Control: Learning Behaviors by Latent Imagination (Dreamer)*. ICLR, 2020.